

Week 3: Integration and Raw Extraction

Hybrid QML and Quantum String Algorithms for Document Extraction

Pushkar Kumar • Mentor: Potluri Krishna Priyatham

Deliverable as specified in the project brief: Working Python script extracting text from sample documents.

1 Objective

Connect the quantum vision output to a digital text stream: interface the quantum-processed features with a lightweight classical OCR backend, and produce a script that takes a document image and returns extracted text.

2 Pipeline

Nine stages, of which two are quantum:

#	Stage	Description
1	preprocess	median filter, Otsu threshold, speckle removal
2	segment	projection profiles $\rightarrow$ character boxes
3	crop	pad to square, resize to 8×8 , normalise
4–5	quantum	R_Y encoding + quanvolutional layer $\rightarrow$ 160 features
6	classify	multinomial logistic regression $\rightarrow$ characters
7	assemble	rebuild lines, insert spaces
8	locate	identify the document identifier field
9	quantum	Grover comparator search (Week 4)

The classifier at stage 6 is the “lightweight classical OCR back-end” the brief specifies: feature extraction is quantum, the decision layer is not.

3 Classical Front End

Three preprocessing operations, each addressing a specific failure.

Median filter removes isolated speckle while preserving edges, unlike a Gaussian blur which would smear thin strokes.

Otsu threshold selects the intensity cut maximising between-class variance. A fixed threshold was rejected because the five tiers differ greatly in overall brightness and any constant fails on at least one.

Speckle removal drops connected components too small to be a stroke. This is not cosmetic: without it the noisy tiers place sub-threshold pixels in *every* row of the page, so line detection found a single 420-row “line” and segmentation collapsed entirely.

Segmentation then sums ink along rows to find text lines, and within each line along columns to find characters. The row threshold is adaptive—a real text line marks a meaningful fraction of the page width—because a fixed “more than one pixel” test allowed residual noise to merge the page into one line on degraded scans.

3.1 Two Fixes Worth Recording

Aspect ratio. Each character box is padded to square *before* resizing to 8×8 . Resizing directly destroys aspect ratio, and for OCR this is not cosmetic: a colon is two small dots in a narrow tall box, and stretched to a square it becomes two thick horizontal bands closely resembling an E. Before this was fixed, $C \rightarrow :$ and $E \rightarrow :$ confusions dominated the error and CER was 32%.

Merged glyph splitting. On low-DPI scans, downsampling plus blur closes the gap between adjacent glyphs so the column profile sees no separator. The obvious fix—cutting a too-wide box into equal slices—was tried and was *wrong*: on degraded tiers it over-segmented badly, cutting single characters apart and pushing error above 100% through pure insertions. The working version cuts only at genuine valleys in the ink profile and leaves a box intact if it has none, so a legitimately wide M survives.

4 Results

Table 1: End-to-end extraction, all 12 documents per tier.

Tier	CER	Segmentation ratio	ID recovered exactly
clean_digital	6.6%	0.95	100%
clean_scan	8.5%	0.93	58%
noisy_scan	79.5%	0.34	0%
clear_handwriting	51.8%	0.70	0%
degraded_handwriting	85.7%	0.99	0%

Two metrics are reported because they diverge. Character error rate measures average character quality; identifier recovery measures whether the pipeline did its actual job, for which one wrong character makes the result useless. `clean_scan` reads at 8.5% CER—visually almost clean—yet yields a usable identifier in only 58% of documents.

Three of five tiers fail outright. Noisy scans and both handwriting tiers have character error rates of 52–86% and recover no identifiers at all. The pipeline is usable on clean digital input and not usable on degraded input, and the tier-by-tier breakdown is reported rather than an average that would obscure it.

5 The Largest Error Source Was a Data Bug

The biggest single improvement in the entire project came from a data-handling defect, not from any modelling change.

How it surfaced. Switching the feature stage from the 64-dimensional marginal readout to the 160-dimensional $+ZZ$ readout *raised* isolated character accuracy but made end-to-end CER *worse*. A strictly better feature set producing strictly worse output is impossible if both are measured on the same input distribution—so they were not.

The cause. Training crops were cut from exact glyph bounds recorded during dataset generation; inference crops come from projection segmentation. These are measurably different distributions: on `clean_scan`, ground-truth boxes have median size 16×19 px against segmented boxes at 12×15 px. The higher-capacity features fitted the training distribution more tightly and transferred less well.

The fix. Training crops are now built by running the real segmentation path over training pages and labelling each detected box from ground truth by overlap, with ambiguous detections dropped rather than guessed.

Table 2: Effect of correcting the train/serve skew.

Tier	CER before	CER after	Reduction
clean_digital	42.9%	6.6%	6.5×
clean_scan	29.7%	8.5%	3.5×
clear_handwriting	74.5%	51.8%	1.4×

5.1 A Caution About the Accuracy Figure

Training on segmented crops *lowers* reported held-out character accuracy from 92.6% to 67.7%. This is not a regression. The two numbers are measured on different crop distributions, and the lower one is measured on the harder, realistic distribution.

The comparison to draw: a model scoring 92.6% on clean bounds produced 42.9% document CER, while a model scoring 67.7% on realistic bounds produced 6.6%.

Isolated-character accuracy was an actively misleading proxy for end-to-end accuracy, and optimising it drove the pipeline in the wrong direction. Any comparison of feature extractors on pre-cut crops—including the Week 2 comparison—measures something narrower than pipeline quality. This is the most transferable finding in the project.

6 Where the Remaining Error Lies

On clean tiers, segmentation recovers 93–95% of characters and isolated character accuracy approaches 99%, yet CER is 6.6–8.5%. The residual is therefore not classification but missed or merged glyphs and word-spacing reconstruction.

The classical front end, not either quantum stage, is the accuracy bottleneck on clean documents.

7 Deliverable Status

Complete. A working script takes a document image and produces extracted text, run successfully across all five tiers.

Artifacts: `integration/pipeline.py`, `notebooks/04_full_pipeline.ipynb`

Usage: `python integration/pipeline.py -doc data/processed/clean_scan/doc_000.png -pattern 38`